

Graph Mining Course Project — Progress Report

Submission Date: 07/01/2026

Course: Graph Mining 4041

Instructor: Dr. Zeinab Maleki

Project Title: An Active Diffusion Neural Network for Graphs (ADGNN)

Student Information

Student Name(s): Sara Tadayon & Pouria Dianati

Student ID(s): 40218903 & 40008773

Email(s): [SaraTadayon](#) & [PouriaDianati](#)

Executive Summary

Since submitting our proposal, we have completed the first implementation milestone: a standard two-layer GCN baseline and an experiment harness suitable for fair comparisons with the simplified ADGNN model. In the proposal, we planned to (1) implement a two-layer GCN baseline, (2) add a diffusion-based baseline, and (3) implement a simplified ADGNN layer, then compare accuracy and over-smoothing behavior. We implemented a two-layer GCN in PyTorch Geometric (GCNConv $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ GCNConv) and verified end-to-end training on Cora, Squirrel, and Cornell using the same pipeline. This slightly deviates from the proposal's initial dataset list (Cora/CiteSeer/PubMed), but keeps the same semi-supervised node classification task while aligning with ADGNN's reported benchmarks that include Squirrel and Cornell. To match the ADGNN evaluation setup, we implemented per-class stratified splits of 48% train, 32% validation, and 20% test, and we report mean $\pm$ std over five initialization seeds. Next, we will implement the proposal's diffusion baseline and the simplified ADGNN layer, then run controlled comparisons and over-smoothing analysis under shared training settings.

Progress on Objectives

Below we report progress aligned with the proposal's three objectives.

Objective 1: Implement a two-layer GCN and one diffusion-based GNN baseline for semi-supervised node classification.

Completed (GCN baseline):

- Implemented a two-layer GCN using PyTorch Geometric, consistent with the proposal's baseline plan.
- Training follows the semi-supervised, transductive setup described in the proposal: logits are computed for all nodes, while cross-entropy loss is applied only to labeled training

nodes via a boolean mask. This matches the “semi-supervised node classification” focus and supports fair baseline comparisons.

- Added validation-based model selection and early stopping (patience-based) to control experiment time, consistent with the proposal’s resource plan.
- Implemented multi-run reporting (five seeds) and mean $\pm$ std aggregation to match the proposal’s evaluation plan (“averaging results over several random seeds”).

Not completed yet (diffusion baseline):

- The proposal specifies adding a diffusion-based baseline (GDC-style) using a fixed personalized-PageRank or heat-kernel propagation matrix. This component is planned next.

Objective 2: Design and implement a simplified ADGNN layer with multiple external source terms.

Completed:

- We implemented and validated the simplified ADGNN layer described in the proposal (ego, boundary-aware, anomaly-sensitive Laplacian term, and normalized adjacency diffusion term with learnable scalar weights).

Objective 3: Compare ADGNN with baselines on citation graphs in terms of accuracy and over-smoothing behavior.

Partially completed (ADGNN vs GCN ready).

- Because both the ADGNN model and the two-layer GCN baseline share the same loaders, masks, early stopping, and multi-seed reporting, we can already run controlled comparisons between ADGNN and GCN under shared training settings, exactly as stated in the proposal. Over-smoothing analysis (cosine similarity between embeddings of different labels versus effective diffusion depth, plus optional visualization) is prepared conceptually but will be finalized after adding the diffusion baseline so that all three models (GCN, diffusion baseline, ADGNN) are evaluated with the same analysis script.

Work Accomplished

Dataset Preparation and Analysis

We implemented dataset loading in Python using PyTorch and PyTorch Geometric, as planned. While the proposal initially listed Cora/CiteSeer/PubMed, our current notebook pipeline runs on Cora, Squirrel, and Cornell to better match the ADGNN paper’s node classification benchmarks (which include Squirrel and Cornell) and to cover both more homophilic and more heterophilic graph settings. The task remains semi-supervised node classification, and the evaluation protocol is consistent. To ensure fair comparisons later (GCN vs diffusion baseline vs ADGNN), we implemented a single split protocol for all models: per-class stratified splits of 48% train,

32% validation, and 20% test. This aligns with the ADGNN paper’s node classification experimental setup (H2GCN split protocol). Splits are fixed across runs (split seed fixed) so repeated-run variance reflects initialization/training randomness rather than resampling.

Dataset	Cora	Squirrel	Cornell
Type	Citation networks	Wikipedia networks	WebKB networks
Homophily ratio	0.81	0.22	0.3
# Nodes	2,708	5,201	183
# Edges	5,429	198,493	295
# Node features	1,433	2,089	1,703
# Classes	7	5	5

Implementation Details

Two-layer GCN: We implemented the proposal’s “two-layer GCN as a passive diffusion baseline” using PyG’s GCNConv layers.

Model architecture:

- Layer 1: neighbor aggregation + learnable linear transform, followed by ReLU
- Dropout: applied between layers for regularization (training only)
- Layer 2: outputs logits for each node

Training setup (semi-supervised, transductive):

- Each epoch computes logits for all nodes, but the loss is computed only on training nodes via the train mask. (`loss = cross_entropy(logits[train_mask], y[train_mask])`)

Reproducibility and reporting:

- We use deterministic seeding per run to make results reproducible. (`seed(seed); manual_seed(seed); cuda_manual_seed_all(seed)`)
- We run 5 seeds (0–4) and report mean $\pm$ standard deviation, matching the proposal’s evaluation plan.

Validation-based selection and early stopping:

- We track validation accuracy each epoch, save the best model state, and restore it before test evaluation. (`if val_acc improves: best_state = state_dict()`)

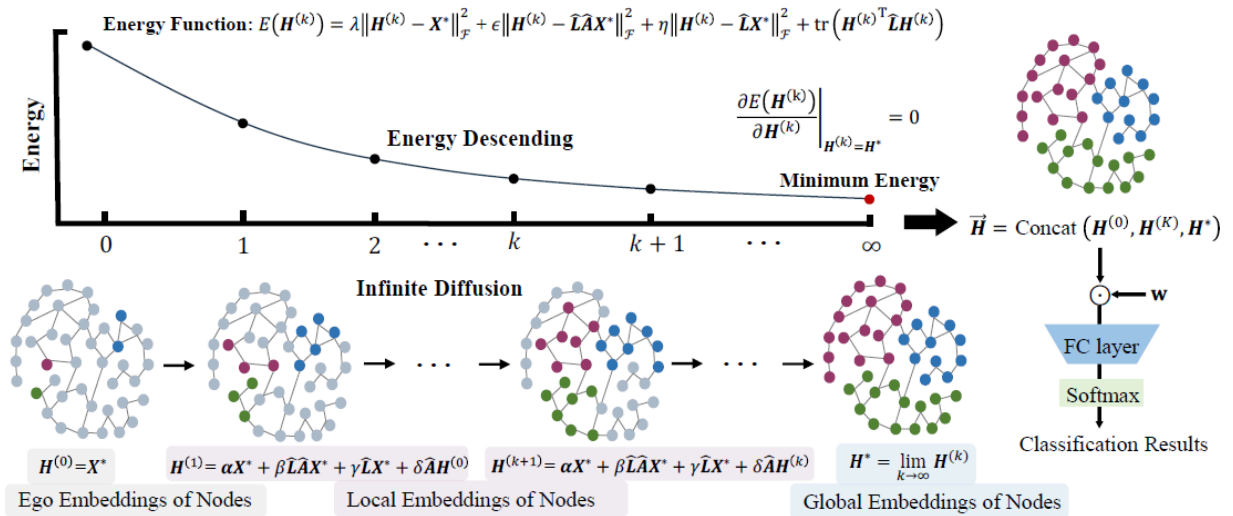
- We use patience-based early stopping to control runtime (stop when validation does not improve for a fixed number of epochs).

Final evaluation:

- After training, we reload the best validation checkpoint and compute test accuracy using the test mask. (`load_state_dict(best_state); test_acc = accuracy(logits, y, test_mask)`)

ADGNN: We implemented ADGNN as a sparse diffusion-based node encoder with a local-global, multi-scale representation.

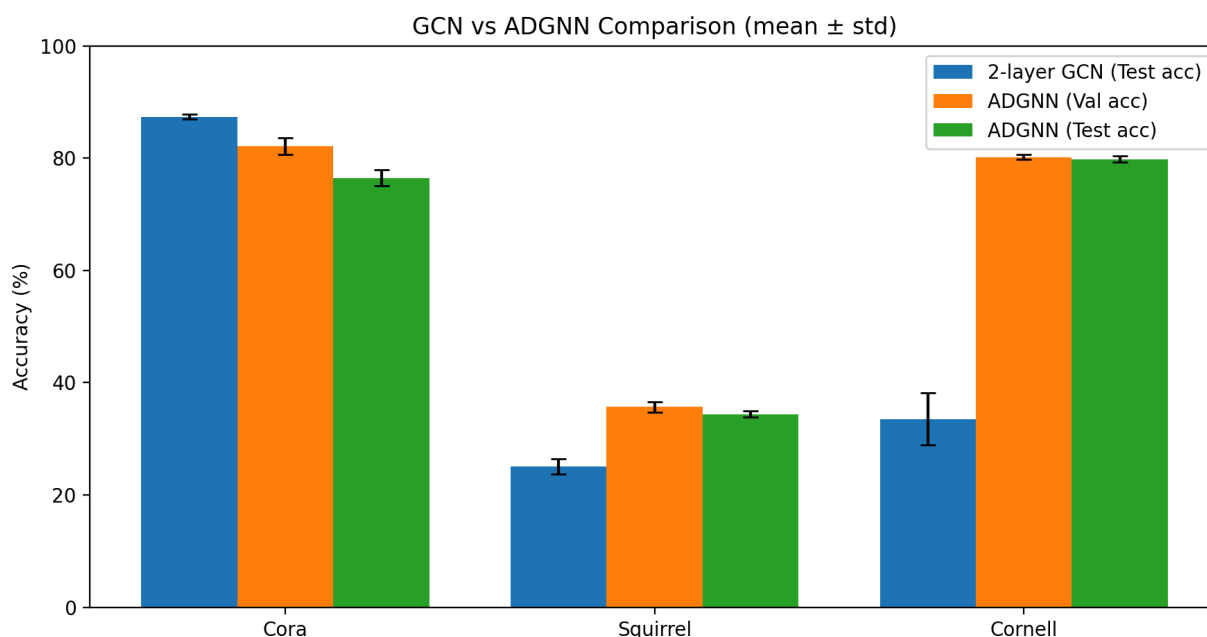
- Ego embedding: apply a learnable linear projection + nonlinearity to node features to get the base embedding.
- Source terms: compute multiple graph-derived signals from the ego embedding and combine them with constrained coefficients (diffusion strength chosen directly; remaining weights set by ratios).
- Local diffusion: iterate a fixed number of update steps that mix the constant source with propagated hidden states to produce the local embedding.
- Global diffusion: compute a long-range embedding via an iterative inverse approximation using repeated propagation and accumulation (no dense inversion).
- Multi-scale + head: concatenate ego, local, and global embeddings and feed them to the paper's classifier head, with an optional large-graph variant controlled by a flag.



Preliminary Results

Across the fixed 48/32/20 stratified split (1 split) and 5 initialization seeds, the 2-layer GCN performs strongly on the homophilous Cora dataset (87.37 ± 0.41 test acc), but struggles on the more heterophilous Squirrel and Cornell (25.09 ± 1.34 and 33.50 ± 4.64). In contrast, your ADGNN (with hyperparameters selected by best mean validation accuracy) achieves notable gains on Squirrel and Cornell (34.41 ± 0.58 and 79.82 ± 0.59 test acc), while it is weaker than GCN on Cora (76.50 ± 1.37 test acc). This pattern is consistent with ADGNN’s design goal of leveraging diffusion with additional source terms to better handle challenging neighborhood mixing (often present in low-homophily graphs), while a standard GCN can remain very competitive on classic citation networks.

Dataset	2-layer GCN (Test acc, mean $\pm$ std)	ADGNN Best Config (K, hd, delta, T, lr, wd)	ADGNN (Val acc, mean $\pm$ std)	ADGNN (Test acc, mean $\pm$ std)
Cora	87.37 ± 0.41	K=4, hd=128, delta=0.6, T=10, lr=0.01, wd=0.0005	82.11 ± 1.47	76.50 ± 1.37
Squirrel	25.09 ± 1.34	K=0, hd=32, delta=0.2, T=5, lr=0.01, wd=0.0005	35.70 ± 0.92	34.41 ± 0.58
Cornell	33.50 ± 4.64	K=2, hd=128, delta=0.6, T=10, lr=0.01, wd=0.0005	80.19 ± 0.40	79.82 ± 0.59



Challenges Encountered and Resolutions

Challenge 1: Keeping experiments manageable under time/hardware limits.

The proposal anticipated experiment-time constraints and recommended early stopping on a GPU-equipped machine.

Solution: We integrated patience-based early stopping driven by validation accuracy and restored the best checkpoint before computing test accuracy, so training remains efficient and reporting remains consistent.

Challenge 2: Preparing for active diffusion complexity without breaking the baseline harness.

The proposal identified stability and efficiency as a key challenge for implementing the active diffusion update rule and suggested validating terms incrementally.

Solution: We started from a standard GCN implementation, validated the pipeline end-to-end, and built reusable utilities (splits, masks, multi-seed evaluation) so that adding diffusion and ADGNN terms will not require changing the evaluation protocol.

References

1. Jiang, M. (2025). An Active Diffusion Neural Network for Graphs (ADGNN)
2. Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR.
3. Klicpera, J., Weißenberger, S., & Günnemann, S. (2019). Diffusion Improves Graph Learning. NeurIPS.
4. Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop.